

Public Safety Alerts Aggregator & Explorer

MASTER PRESENTATION & SOLUTION ARCHITECTURE

Role: Cybersecurity &
Operations Lead (Critical
Signal)

Team: Ettore Vescio,
Christian Finocchiaro,
Thomas Smith, Pradip
Pandey

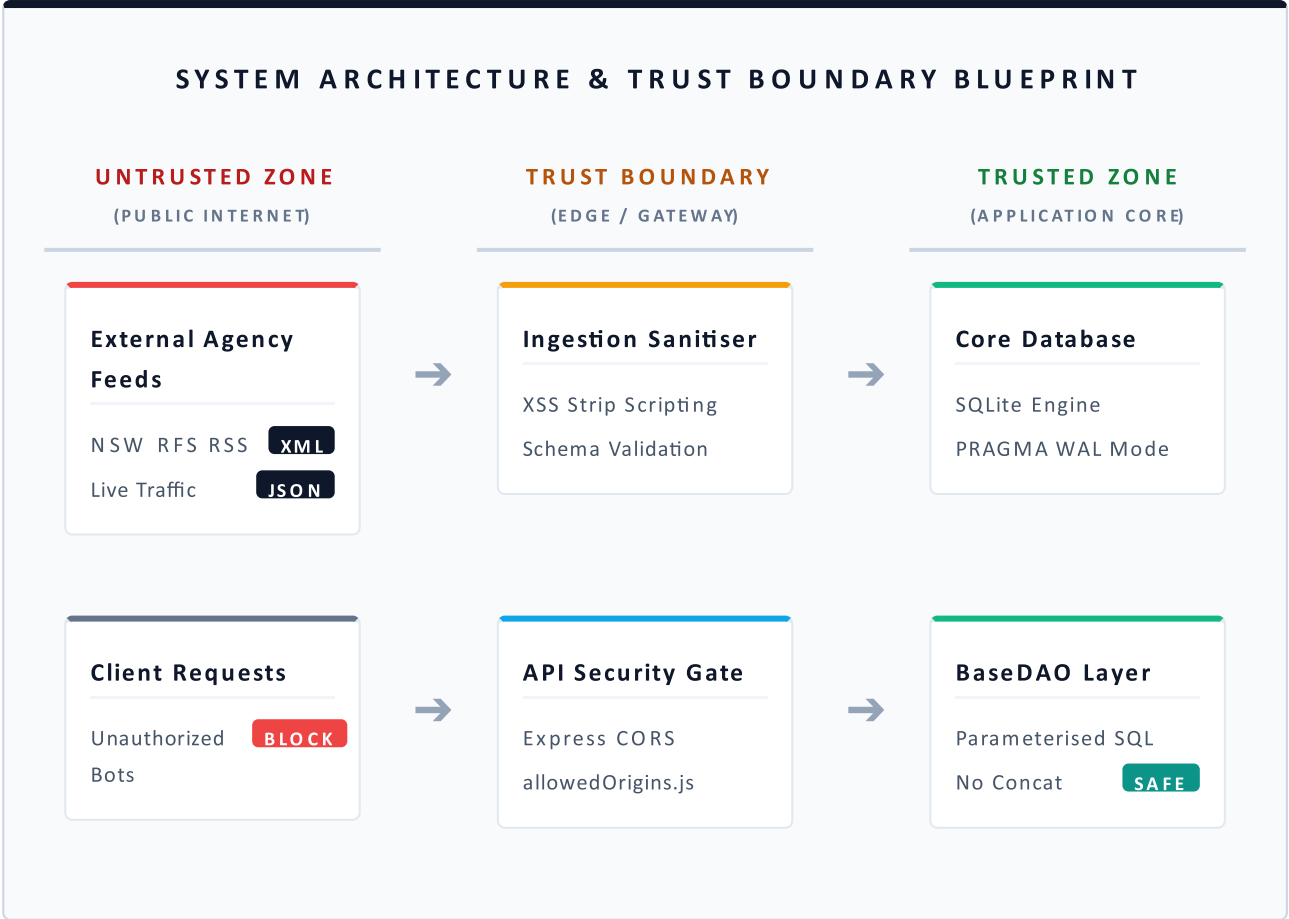
Assessment: A3 — Solution
Architecture & Working
Prototype Walkthrough

1. CYBERSECURITY & OPERATIONS EXECUTIVE SUMMARY

In designing the **Critical Signal** prototype, cybersecurity was established not as a secondary compliance check, but as a foundational architectural pillar. Utilizing a hybrid Agile methodology, we embedded a **NISTaligned risk treatment workflow** into our initial development sprints.

Proactive Posture: By explicitly defining strict **trust boundaries** between external untrusted feeds and our verified application core, we implemented deterministic, automated security controls at every boundary transition.

This approach—automating input verification, enforcing parameterized SQL queries, and utilizing strict origin filters—minimizes systemic vulnerabilities and guarantees continuous uptime, which is vital for a public safety alerting architecture.



2. ARCHITECTURAL DECISIONS & TRADE-OFFS

The development of the prototype was guided by critical design choices aligned to both functional and nonfunctional requirements (NFRs).

DESIGN DECISION	ENGINEERING JUSTIFICATION	REQUIREMENT & TRADE-OFF TRACEABILITY
SQLite DB (Session 1) with MySQL Migration Path	Provides an immediate, zero-config relational setup for rapid prototyping without administrative overhead.	Trade-off: SQLite lacks horizontal scaling. We mitigated this by building fully decoupled DAOs, making the Session 2 transition to MySQL seamless.
Feed Adapter Pattern	Decouples agency-specific payload parsing. If the TfNSW API changes, only one adapter is quarantined, preventing system crashes.	NFR: Maintainability & Reliability. Ensures continuous uptime even during third-party outages.
Parameterised SQL	Strictly prohibits string concatenation in queries. User/feed variables are explicitly bound.	Security NFR: Provides 100% immunity against SQL Injection (SQLi) vulnerabilities.

3. WORKING PROTOTYPE: FEASIBILITY EVIDENCE

The presentation layer features a responsive Web UI built with Next.js 16 and React 19, styled via TailwindCSS, and deeply integrated with the official Google Maps API.

A. Interactive Spatial Exploration

Upon landing, users view a dark-themed dashboard splitting the interface between active alert cards and an interactive spatial map. RFS alerts utilize distinct fire icons, while TfNSW events render as high-visibility warning pins.

[VISUAL EVIDENCE PLACEHOLDER: WEB UI DASHBOARD]

Demonstrates Google Maps marker integration and responsive React 19 components.

[public-safety-alerts-aggregator/docs/screenshots/frontend_POC/alert_page_layout.png](#)

B. Relational Ingestion Integrity

To prove our backend translates heterogeneous schemas (XML/JSON) into a normalized structure, we conducted direct database verification tests.

[VISUAL EVIDENCE PLACEHOLDER: SQLITE ALERTS TABLE]

Confirms successful ingestion, de-duplication, and parsing of severity levels.

[public-safety-alerts-aggregator/docs/screenshots/mysql_ingestion_POC_tests/
json_ingestion_test_mysql_alerts_table.png](#)

4. QUALITY ASSURANCE & Q&A PREPAREDNESS

Our comprehensive validation strategy leverages **Jest** and **Supertest** to guarantee system resilience.

VALIDATED CONTROL	IMPLEMENTATION DETAILS	STATUS
SQLi Defense	Parameterised queries tested via BaseDAO wrapper (BaseDAO.test.js).	[PASSED]
XSS Mitigation	HTML descriptions sanitized during ingestion (alertUtilities.test.js).	[PASSED]
Data Minimisation	Adheres to the Australian Privacy Act by strictly hashing records and storing no physical addresses or PII.	[VERIFIED]